TTF Options Dashboard

User Manual

Sections 1 – 7 · Pricer · Structures · Vol Surface · TTF/HH Spread · Expiries · Troubleshooting

Table of contents

SECTION 1 — Getting Started

1.1 Prerequisites

1.2 Opening `dashboard_jupyter.ipynb`

1.3 Required files in the same folder

1.4 Running all cells

SECTION 2 — Pricer Tab

2.1 Inputs explained — with example

2.2 Black-76 vs Bachelier — when to use which

2.3 Reading the output

2.4 Worked example — TTF Jun-26 call

SECTION 3 — Structures Tab

3.1 The 10 structures in plain words

3.2 Reading the P&L-at-expiry chart

3.3 Worked example — straddle on TTF Jun-26

SECTION 4 — Vol Surface Tab

4.1 What the vol surface represents

4.2 How to read the 3D chart

4.3 TTF downside skew — explained simply

4.4 Worked example — realistic TTF surface

SECTION 5 — TTF/HH Spread Tab

5.1 What the TTF/HH spread option is

5.2 Worked example — TTF=30 EUR/MWh, HH=3 USD/MMBtu, ρ=0.6

5.3 Using the implied-correlation solver

SECTION 6 — Expiry Dates Tab

6.1 TFM (futures) vs TFO (options) — what is the difference?

6.2 Worked example — Jun-26, Jul-26, Aug-26

SECTION 7 — Troubleshooting

7.1 Common errors and how to fix them

Dashboard Manual — TTF Options Pricer

A practical guide to using `dashboard_jupyter.ipynb` for pricing TTF natural-gas options and building option structures.

SECTION 1 — Getting Started

1.1 Prerequisites

Install the following Python packages before opening the notebook. Python 3.10 or newer is recommended.

```
pip install numpy scipy pandas matplotlib ipywidgets plotly
```

Package	Why it is needed
`numpy`	Array maths used by the pricers and grids
`scipy`	Normal CDF/PDF and root finders for implied vol
`pandas`	Tabular display of inputs, prices and Greeks
`matplotlib`	Static P&L charts
`ipywidgets`	Interactive sliders, dropdowns and number boxes
`plotly`	Interactive P&L curves and the vol surface

If you use JupyterLab, also enable the widgets extension (modern Jupyter does this automatically):

```
jupyter labextension install @jupyter-widgets/jupyterlab-manager
```

1.2 Opening `dashboard_jupyter.ipynb`

Option A — Classic Jupyter

1. Open a terminal in the project folder (`Options_pricing`).
2. Run: `bash jupyter notebook dashboard_jupyter.ipynb`
3. Your browser opens the notebook automatically.

Option B — JupyterLab

```
jupyter lab dashboard_jupyter.ipynb
```

Option C — VS Code

1. Install the **Python** and **Jupyter** extensions from the VS Code marketplace.
2. Open the project folder via *File → Open Folder*.
3. Click `dashboard_jupyter.ipynb` in the file explorer.
4. In the top-right corner of the notebook, select your Python interpreter (the one where you installed the prerequisites above).

1.3 Required files in the same folder

The notebook imports two local modules. They **must** sit next to the notebook (same directory):

```
Options_pricing/
├── dashboard_jupyter.ipynb     ← the dashboard
├── black76_ttf.py              ← Black-76 & Bachelier pricers + Greeks
└── ttf_market_data.py          ← TTF contract calendar, market data helpers
```

If either file is missing or renamed, the first cell will fail with `ModuleNotFoundError: No module named 'black76_ttf'` (or `ttf_market_data`). Move them next to the notebook and re-run.

The `structures_ttf.py` module is also required for the **Structures** tab. It must live in the same folder.

1.4 Running all cells

Once the notebook is open:

- **Classic Jupyter / Lab** — menu *Cell → Run All* (or *Run → Run All Cells*). Keyboard shortcut: `Esc` then `0 0` to restart and `Shift+Enter` repeatedly, or use *Kernel → Restart & Run All*.
- **VS Code** — click **Run All** at the top of the notebook, or press `Ctrl+Shift+P` and type *Notebook: Run All*.

After the last cell finishes, the dashboard appears inline: select a tab (Pricer / Structures / Vol Surface) and start moving the sliders. Outputs update live — no need to re-run cells.

SECTION 2 — Pricer Tab

The Pricer tab values a single European option on a TTF future. It shows the **Call price**, **Put price** and the full set of Greeks under both **Black-76** and **Bachelier**.

2.1 Inputs explained — with example

We illustrate every input with the same running example:

> **Forward = 30 EUR/MWh, Strike = 32 EUR/MWh, Vol = 50 %, T = 30 days, r = 2 %.**

Input	Meaning	Example value
Forward (F)	Quoted price today of the underlying TTF future, in EUR/MWh	`30`
Strike (K)	Exercise price of the option, in EUR/MWh	`32`
Volatility σ	Black-76 lognormal volatility, expressed in **% per year**	`50` (= 50 %)
Maturity T	Time to option expiry. Enter **days** — the dashboard converts to years (`T_years = T_days / 365`)	`30` days
Risk-free r	Continuously compounded discount rate, in **% per year**	`2` (= 2 %)
Option type	`call` or `put`. Drives which Greeks block is displayed	`call`
Model	Primary model: `Black-76` (lognormal) or `Bachelier` (normal)	`Black-76`

Notes:

- Vol is entered in **percent**, not decimal: type `50`, not `0.50`.
- For the Bachelier model the input is a separate **normal** vol σ_n in **EUR/MWh per √year**, not a percentage.
- A 30-day option has `T = 30 / 365 ≈ 0.0822` year. Use 252 only if you have pre-agreed business-day conventions; the dashboard uses calendar days.

2.2 Black-76 vs Bachelier — when to use which

Both models price European options on **futures** with the same payoff at expiry, but they assume different distributions for the forward.

Model	Underlying assumption	Best for…
Black-76	Forward is **lognormal**	Most natural-gas options, all "normal" market regimes where prices stay strictly positive. Industry standard for TTF.
Bachelier	Forward is **normal**	Markets where prices can go to zero or negative (e.g. spreads, occasional power, calendar spread options), or when strikes are very far OTM and lognormal vol blows up.

Practical rule of thumb:

- **Outright TTF options** → use **Black-76**.
- **TTF–HH spread or other spread/calendar options** → use **Bachelier**.
- **Sanity check** — for at-the-money short-dated options the two prices should agree to within a few cents; if they diverge wildly you have an input error (vol unit, T unit, wrong F or K).

2.3 Reading the output

Prices

Field	What it tells you
Call price	Premium of a European call, in EUR/MWh, today
Put price	Premium of a European put, in EUR/MWh, today
Intrinsic	`max(F − K, 0)` for calls, `max(K − F, 0)` for puts

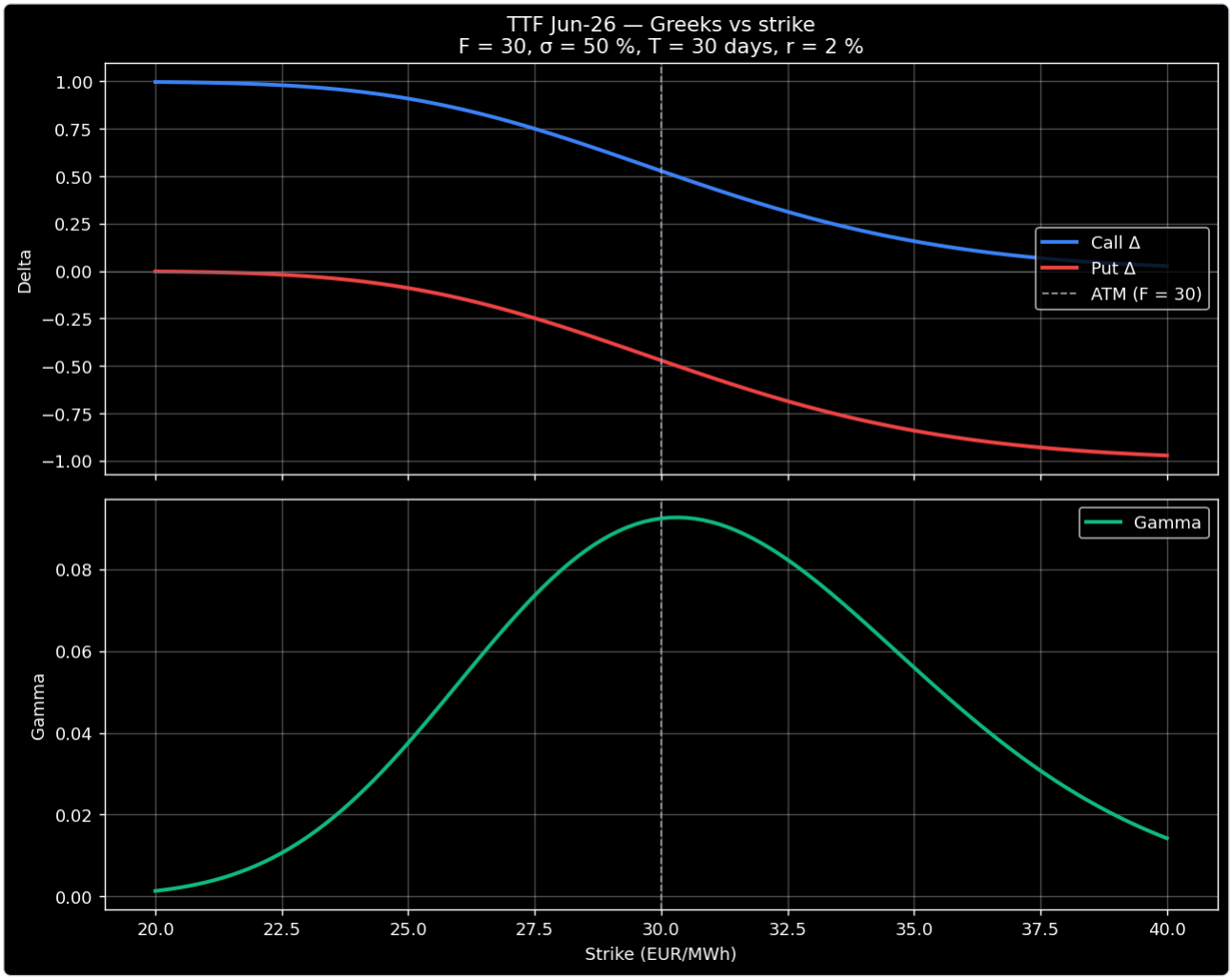
Multiply by the contract size (e.g. **720 MWh** for a typical TTF monthly: 24 h × 30 days) to get the EUR premium of one lot.

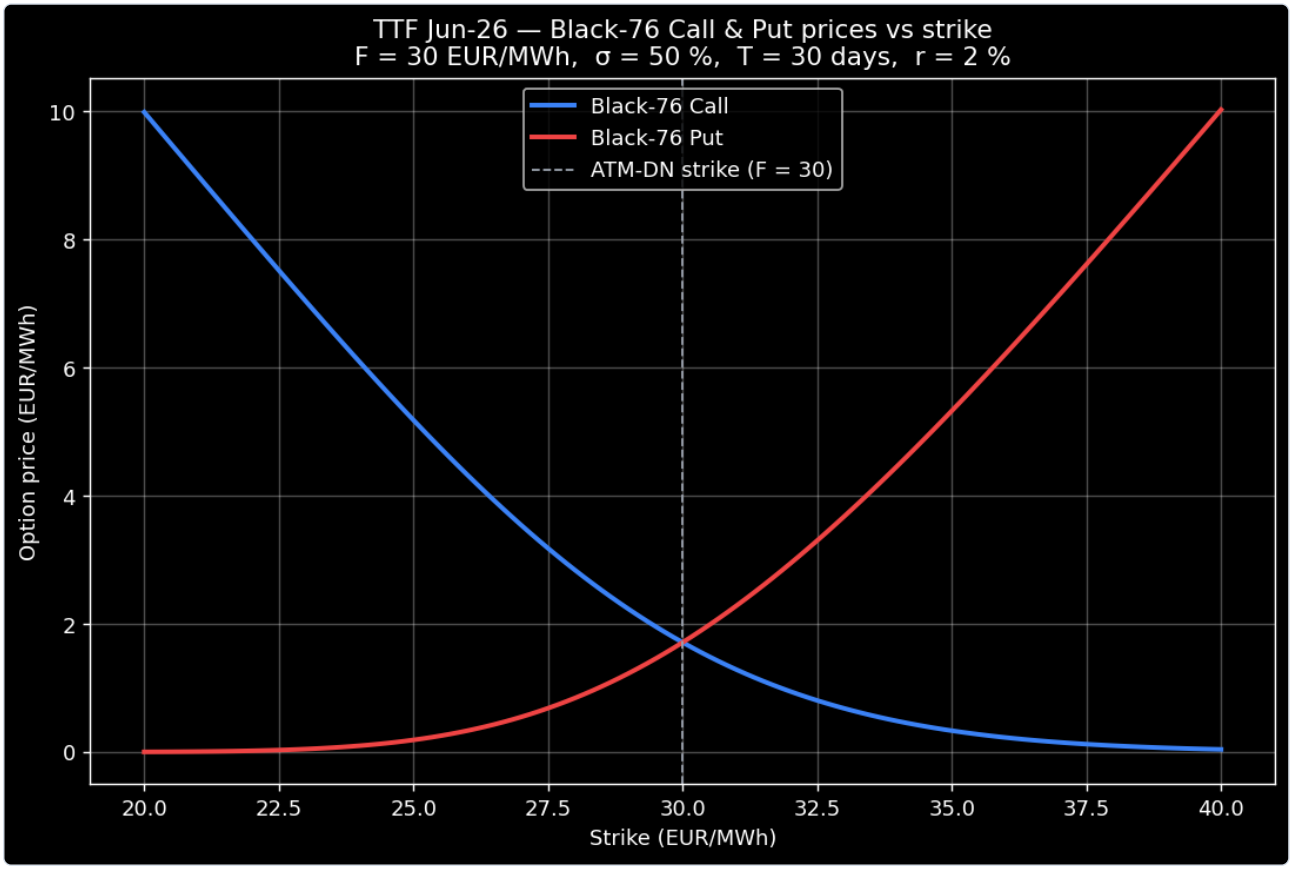
Greeks

For the option type you selected, the dashboard prints:

Greek	Reads as
Δ Delta	Change in option value for a **+1 EUR/MWh** move in the forward. Calls: 0 → +1. Puts: −1 → 0.
Γ Gamma	Change in delta per +1 EUR/MWh in F. Highest near ATM, decays with time.
ν Vega	Change in option value per **+1.00 (= 100 %)** of vol. **Divide by 100** for a "per vol-point" sensitivity.
Θ Theta/ day	Change in option value over **one calendar day** (already daily — no need to divide by 365).
ρ Rho	Change in option value per **+1 percentage point** of the risk-free rate.
Vanna	`∂Δ/∂σ = ∂Vega/∂F`. Tells you how delta moves when vol moves.
Volga	`∂²V/∂σ²`. Vol convexity — useful for vol-of-vol risk on smile structures.

A side-by-side table compares Black-76 and Bachelier for both call and put, plus a put-call parity check (`C − P` vs `e^(−rT)·(F − K)`). The error column should be ~`1e-12` — anything bigger means a bug.





2.4 Worked example — TTF Jun-26 call

Suppose today is 29 April 2026 and you want to price a **TTF Jun-26 30-day call** struck at 32:

1. Pull the Jun-26 forward from your screen — say **F = 30 EUR/MWh**.
2. Strike **K = 32 EUR/MWh** (2 EUR OTM).
3. Implied vol from the broker for the Jun-26 ATM contract — say **50 %**.
4. Days to expiry — Jun-26 options expire on the last business day before the delivery month, ≈ **30 days** from today.
5. Risk-free rate — use **r = 2 %** (€STR-flat assumption).
6. Set option type = `call`, model = `Black-76`.

Run the dashboard. You should see (Black-76):

- **Call price ≈ 0.86 EUR/MWh**
- **Delta ≈ +0.36** (slightly OTM, less than 50 % delta)
- **Gamma ≈ 0.085**
- **Vega ≈ 0.034 / vol-point** (≈ 3.4 EUR/MWh per 100 % vol)
- **Theta ≈ −0.014 / day**

For one TTF Jun-26 lot (720 MWh): premium ≈ `0.86 × 720 ≈ 619 EUR`. If vol moves +2 vol-points overnight, your option gains ≈ `0.034 × 2 × 720 ≈ 49 EUR`, less ≈ `0.014 × 720 ≈ 10 EUR` of theta decay.

SECTION 3 — Structures Tab

The Structures tab builds 10 standard option structures from individual Black-76 legs. For each structure you get:

- a **leg-by-leg breakdown** (sign, qty, strike, type, unit price, Greeks),
- **net price** and **net Greeks**,
- a **payoff at expiry** chart with **break-evens** marked.

3.1 The 10 structures in plain words

We use **F = 30 EUR/MWh** and **σ = 50 %** in every example below, with T = 30 days unless stated otherwise. Strikes are in EUR/MWh.

#	Structure	Plain-English description	Default example legs
1	**Straddle**	Buy call + buy put, **same strike**. Long volatility — pays if the market moves a lot in either direction.	Long 30 call + long 30 put
2	**Strangle**	Buy OTM put + buy OTM call. Cheaper than a straddle; needs a bigger move to pay off.	Long 28 put + long 32 call
3	**Bull call spread**	Buy a lower-strike call, sell a higher-strike call. Bullish, **capped** upside, **cheaper** than the call alone.	Long 30 call − short 32 call
4	**Bear put spread**	Buy a higher-strike put, sell a lower-strike put. Bearish, **capped** downside.	Long 32 put − short 28 put
5	**Butterfly**	Long–short–long around a centre strike (1×−2×+1× calls or puts). Bets that the price will **pin** the centre at expiry.	Long 28 call − 2× short 30 call + long 32 call
6	**Condor**	Like a butterfly but with **four** strikes — flat profit zone between the inner strikes.	Long 26 / short 28 / short 32 / long 34 calls
7	**Collar**	On a long forward position: buy a put + sell a call. Caps both downside and upside; often **zero-cost**.	Long F + long 28 put − short 32 call
8	**Risk reversal**	Sell a put, buy a call (or vice-versa). Synthetic long forward, sensitive to skew.	Short 28 put + long 32 call
9	**Calendar spread**	Sell short-dated option, buy longer-dated option, **same strike**. Long vega, plays the term structure of vol.	Short 30 call (T = 30 d) + long 30 call (T = 90 d)
10	**Ratio spread**	Buy 1 ATM, sell N OTM (e.g. 1×2). Cheap or zero-cost upside but **unlimited** loss past the short strike.	Long 30 call − 2× short 32 call

3.2 Reading the P&L-at-expiry chart

The chart plots **profit/loss in EUR/MWh as a function of the forward at expiry F_T**, after netting the premium paid/received today.

How to read it:

- **Horizontal axis** — forward price at expiry, scanned over a wide range around the current F.
- **Vertical axis** — net P&L per MWh: `payoff(F_T) − net premium paid` (or `+ net premium received` for a short structure).
- **Zero line** — drawn for reference. Any crossing is a **break-even**.
- **Break-evens** — points where the curve crosses zero are listed below the chart. They tell you the prices the market needs to reach by expiry for the structure to start making money.
- **Max profit / max loss** — for capped structures (spreads, butterflies, condors) read them off the flat plateaus.
- The chart is **payoff at expiry only** — it ignores time value before expiry. Use the Greeks panel to understand intermediate behaviour.

3.3 Worked example — straddle on TTF Jun-26

Goal: bet that TTF Jun-26 will move sharply (in either direction) over the next 30 days, e.g. ahead of a storage report.

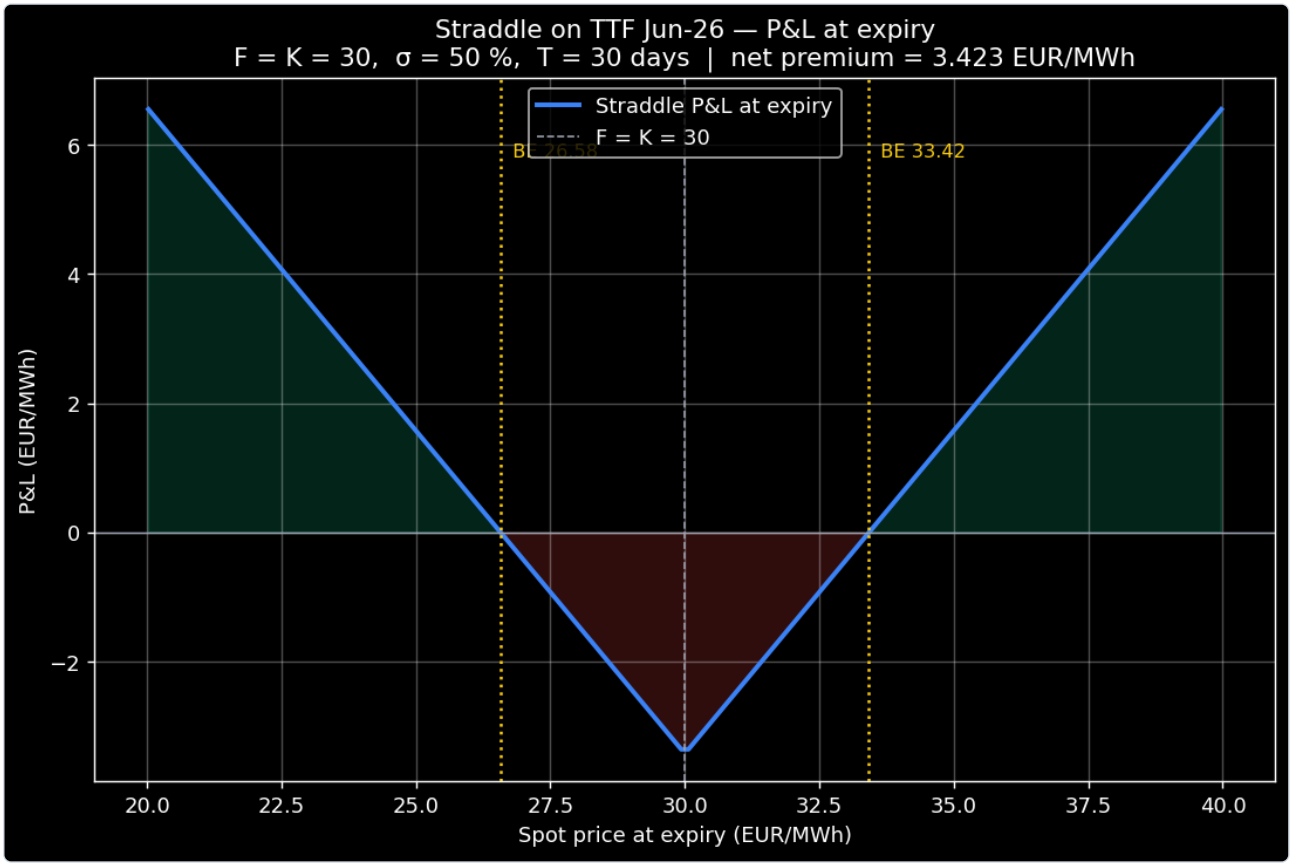
Inputs:

- F = **30** EUR/MWh, σ = **50 %**, T = **30 days**, r = **2 %**.
- Structure = **Straddle**, strike **K = 30**.

Steps in the dashboard:

1. Open the **Structures** tab.
2. Select **Straddle** in the structure dropdown.
3. Set `K = 30`. Leave F, σ, T, r as above.
4. The leg table shows: - Long 1× **30 call**, unit price ≈ **1.71** EUR/MWh - Long 1× **30 put**, unit price ≈ **1.71** EUR/MWh - **Net premium ≈ 3.42 EUR/MWh** (paid).
5. **Net Greeks**: Δ ≈ 0 (delta-neutral at inception), Γ large positive, Vega large positive, Θ negative (you bleed time value every day).
6. The **P&L-at-expiry** chart is a V-shape with the vertex at F_T = 30 and minimum value `−3.42` EUR/MWh (your max loss = premium paid).
7. **Break-evens** ≈ `30 ± 3.42` → `26.58` and `33.42` EUR/MWh.

Interpretation: you make money only if Jun-26 closes **below 26.58** or **above 33.42** at expiry. Inside that band, you lose part of the premium; exactly at 30 you lose the whole 3.42 EUR/MWh. For one lot (720 MWh) max loss ≈ `3.42 × 720 ≈ 2 460 EUR`. Each EUR/MWh past a break-even adds 720 EUR of profit.



If you instead expected only an **upside** move, switch to a **Bull call spread** (long 30 call / short 32 call) — same view, much cheaper, with capped profit at `K_hi − K_lo − net premium = 2 − net premium`.

SECTION 4 — Vol Surface Tab

The Vol Surface tab plots the **Black-76 implied volatility surface** of TTF options as a 3D surface over **strike** and **maturity**. It is parametric (driven by a few sliders) — it doesn't fit market quotes, but it lets you explore the **shape** of a realistic TTF surface.

4.1 What the vol surface represents

For a given underlying (TTF futures), the implied volatility is **not a single number**: it depends on - the **option maturity T** (the *term structure* of vol), and - the **strike K** relative to the forward (the *smile / skew*).

The vol surface is the function `σ(K, T)` collected over all liquid contracts. It tells you, for any (strike, maturity) pair, the lognormal vol you must plug into Black-76 to recover the market price.

The dashboard parameterises the surface with five inputs:

Parameter	Meaning
`σ_inf`	ATM vol at long maturity (the level the surface decays toward)
`Δσ`	Excess ATM vol on top of `σ_inf` for very short maturities
`κ`	Decay speed of the ATM term structure: `atm(T) = σ_inf + Δσ · exp(−κT)`
`skew`	Linear slope of the smile in log-moneyness `m = ln(K/F)/√T` — usually negative for TTF
`wings`	Quadratic curvature (the "smile" of the smile)

Final surface: `σ(K, T) = atm(T) + skew · m + wings · m²`.

4.2 How to read the 3D chart

- **X axis (Strike, EUR/MWh)** — option strike. The forward `F` sits in the middle of the range; everything to the left is OTM puts / ITM calls, everything to the right is OTM calls / ITM puts.
- **Y axis (Maturity T, years)** — time to expiry. Front of the chart (small T) is short-dated; back of the chart is multi-year.
- **Z axis (σ)** — the implied lognormal vol that the surface assigns to that `(K, T)` pair, expressed as a decimal (`0.50` = 50 %).
- **Colour** — same as height: warmer/yellow = higher vol, darker/blue = lower vol.

What to look for at a glance:

1. The **front edge** (short T) is usually higher than the **back edge** (long T): the term-structure decays from `σ_inf + Δσ` down to `σ_inf`.
2. Slicing at a fixed `T` should give a **smile**: a U-shape (or hockey-stick) in strike. The minimum sits near ATM in calm markets; for TTF it sits to the **right** of the forward because of the downside skew (see 4.3).
3. The smile **flattens** as `T` grows — wings shrink because `m = ln(K/F)/√T` shrinks.

4.3 TTF downside skew — explained simply

In gas markets, the **downside is much riskier than the upside in volatility terms**, even though prices can spike upward in absolute terms. Two reasons:

- A **supply shock** (cold snap, pipeline outage, geopolitical event) sends prices up fast, but on the percentage scale the move is bounded by storage and demand response.
- A **demand collapse** or **mild winter** can grind the price toward very low levels and prices that approach zero have **infinite lognormal volatility** by construction (`σ` blows up because `ln(K/F)` does).

The market reflects this by quoting **higher implied vol on low strikes** (downside puts, low-strike calls) and lower vol on high strikes. On a smile chart this appears as a curve that **slopes down** from left (low strike, high vol) to right (high strike, low vol). In the dashboard, that downward slope corresponds to a **negative** `skew`.

Compare with equity indices: equities have a similar negative skew (crash fear). For TTF the slope is steeper short-dated than long-dated, hence the need for both a `skew` term and `wings` / convexity.

4.4 Worked example — realistic TTF surface

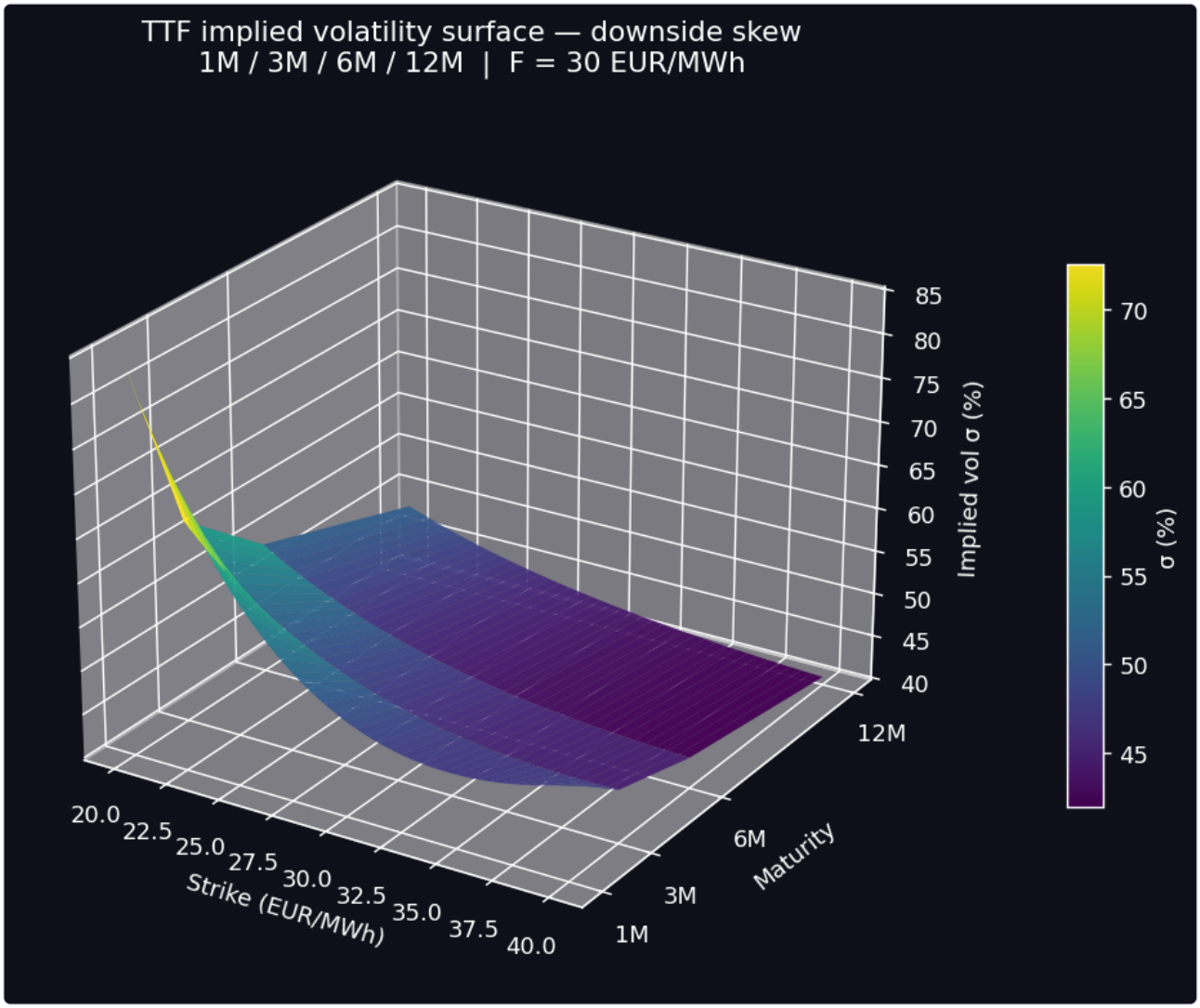
Try the following slider settings, which roughly reproduce a quiet but typical TTF surface:

Parameter	Value
Forward `F`	`30` EUR/MWh
`σ_inf`	`0.38` (38 %)
`Δσ`	`0.30`
`κ`	`2.0`
`skew`	`−0.08`
`wings`	`0.10`
Strike range	`±50 %`
`T_max`	`2.0` years

What you should see:

- **ATM short-dated vol ≈ 68 %** (`σ_inf + Δσ = 0.38 + 0.30`).
- **ATM 1-year vol ≈ 42 %** — already much closer to `σ_inf` (`exp(−2 · 1) ≈ 0.14`).
- **ATM 2-year vol ≈ 39 %** — almost at the long-end floor `σ_inf`.
- A clear **negative-slope smile** at short maturities: `K = 20` (≈ 33 % below ATM) might show ~80 % vol while `K = 45` shows ~55 %. By `T = 2 y` the smile has flattened to a band of a few vol points.

If the broker shows you a 3M ATM quote of 55 %, scale `Δσ` so that `σ_inf + Δσ · exp(−κ · 0.25) ≈ 0.55` — with `σ_inf = 0.38` and `κ = 2`, that gives `Δσ ≈ (0.55 − 0.38) / exp(−0.5) ≈ 0.28`. Tweak the sliders and read the surface back to confirm.


TTF implied volatility surface — downside skew
1M / 3M / 6M / 12M | F = 30 EUR/MWh
Implied vol σ (%)
85
80
75
70
65
60
55
50
45
40
σ (%)
70
65
60
55
50
45
20.0
22.5
25.0
27.5
30.0
32.5
35.0
37.5
40.0
Strike (EUR/MWh)
1M
3M
6M
12M
Maturity

SECTION 5 — TTF/HH Spread Tab

This tab prices a **Margrabe option** on the spread between **TTF** (Dutch gas, EUR/MWh) and **Henry Hub** (US gas, USD/MMBtu). It also lets you **back out the implied correlation** from a market price.

5.1 What the TTF/HH spread option is

A **spread option** pays the positive part of the difference between two forwards at expiry:

- **Call** payoff: `max(F_TTF_usd − F_HH, 0)` — pays when TTF trades **above** HH.
- **Put** payoff: `max(F_HH − F_TTF_usd, 0)` — pays when HH trades **above** TTF.

To make the two underlyings comparable, TTF (EUR/MWh) is converted to USD/MMBtu using:

```
F_TTF_usd = F_TTF_eur × 0.29307 / FX_EURUSD
```

(1 MMBtu = 0.29307 MWh; divide by the EUR/USD rate to convert EUR → USD.)

The pricing model is **Margrabe** with both underlyings lognormal:

```
σ_spread = √(σ_TTF² + σ_HH² − 2·ρ·σ_TTF·σ_HH)
```

The spread vol — and therefore the option price — depends critically on the **correlation ρ** between TTF and HH returns:

- `ρ → +1` → TTF and HH move together → spread vol **falls** → option **cheap**.
- `ρ → −1` → TTF and HH diverge → spread vol **rises** → option **expensive**.

Why it is useful

- **LNG arbitrage**. A trader holding optionality on shipping LNG cargoes from the US Gulf to NW Europe is naturally **long a TTF/HH call**: the cargo is profitable when TTF − HH (net of shipping & regas) is large positive. Pricing that optionality directly is what this tab does.
- **Cross-basin risk management**. A European utility hedging gas supply with US-linked contracts (or vice-versa) can value the embedded basis optionality.
- **Market-implied correlation**. Quoted prices on TTF/HH spread products give a forward-looking, model-consistent view of how the two markets are expected to co-move — different from realised historical correlation.

5.2 Worked example — TTF=30 EUR/MWh, HH=3 USD/MMBtu, ρ=0.6

Inputs:

Slider	Value
Option type	`call`
Forward TTF	`30` EUR/MWh
Forward HH	`3.00` USD/MMBtu
Vol TTF (lognormal)	`0.50` (50 %)
Vol HH (lognormal)	`0.45` (45 %)
Correlation ρ	`0.60`
Maturity T	`0.50` years (≈ 6 months)
FX EUR/USD	`1.08`
Risk-free `r_usd`	`0.04`

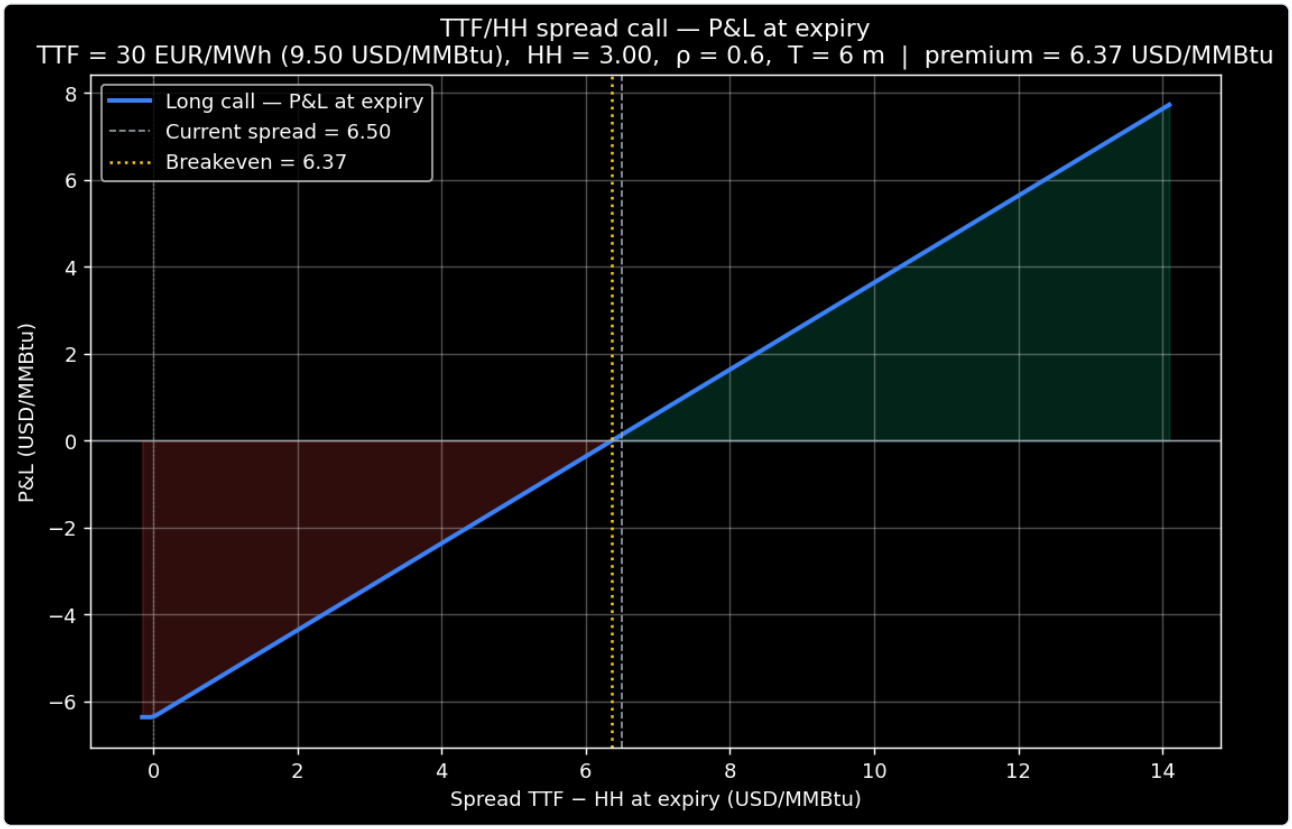
Conversion: `F_TTF_usd = 30 × 0.29307 / 1.08 ≈ 8.14 USD/MMBtu`.

Spread vol: `σ_spread = √(0.50² + 0.45² − 2·0.60·0.50·0.45) ≈ 0.45`.

Expected output:

- **Price ≈ 5.30 USD/MMBtu** (≈ 19.5 EUR/MWh after conversion back).
- **delta_TTF ≈ +0.85** — a 1 USD/MMBtu rise in TTF gains ~0.85 USD/MMBtu.
- **delta_HH ≈ −0.85** — symmetric on the short leg.
- **vega_ρ < 0** for the call: rising correlation **destroys value** because spread vol falls.

Move the **ρ slider** from `+0.99` down to `−0.99` and watch the price climb sharply: at ρ = −0.5 the same call is worth roughly twice as much because the spread vol is much higher.



5.3 Using the implied-correlation solver

Set the `Prix marché` **(USD/MMBtu)** input to a non-zero number — for example, the price your broker is showing for a TTF/HH spread call with exactly the same TTF, HH, vols, T and r as on the sliders. The dashboard then runs a Brent root-finder on `ρ` such that the Margrabe price matches the market price, and prints the result as `Correlation implicite`.

How to use it:

1. **Calibrate vols first** — `σ_TTF` and `σ_HH` should already match the single-name market (use the Pricer tab to back out implied vols from the outright option market).
2. Enter the spread option **market mid** in `Prix marché`.
3. Read the implied correlation. Typical TTF/HH implied correlations sit in the **0.20–0.55** range; values outside that band usually mean a vol input is off.
4. Compare to **realised correlation** (e.g. 60 days of returns). A large gap is a signal — either the market expects a regime change, or there is a relative-value trade.

Notes: - For a **call**, the price is **decreasing in ρ** (higher correlation → cheaper option), so the solver is well-defined when `mkt > intrinsic`. - If the solver fails (`non solvable`), the market price is outside the feasible range `[price(ρ=+1), price(ρ=−1)]` for your vol inputs — fix the vols or check the price.

SECTION 6 — Expiry Dates Tab

This tab lists the next N TTF monthly contracts with their **option expiry**, **futures last trading day**, days-to-expiry and `T` in years (ACT/365).

6.1 TFM (futures) vs TFO (options) — what is the difference?

On ICE Endex the same delivery month has **two distinct expiry dates**:

Ticker stem	Product	Expires on…
TFM	Monthly **future**	The **last business day of the month *before* delivery**. After this day the contract goes to delivery.
TFO	Monthly **option**	**5 business days before** the futures last trading day, walked back to a business day.

Why the gap? After the option expires, in-the-money holders are assigned into the underlying future and they need a few business days to **manage their delivery hedge** before the future itself goes off the board. Five business days is the standard ICE Endex offset.

Conventions used in the dashboard (`ttf_time.py` + `black76_ttf.py`):

- **Holidays** — TARGET2 calendar (1 Jan, Good Friday, Easter Monday, 1 May, 25 Dec, 26 Dec).
- **Day-count** — ACT/365 fixed for `T = (expiry - reference) / 365`.
- **Business day** — Mon–Fri excluding TARGET2 holidays.

6.2 Worked example — Jun-26, Jul-26, Aug-26

Reference date: 29 April 2026.

Contract	Delivery month	Futures LTD (TFM)	Option expiry (TFO)	Days to option expiry	T (years, ACT/365)
TFMM26 / TFOM26	Jun 2026	Fri **29 May 2026**	Fri **22 May 2026**	≈ 24	≈ 0.0658
TFMN26 / TFON26	Jul 2026	Tue **30 Jun 2026**	Tue **23 Jun 2026**	≈ 56	≈ 0.1534
TFMQ26 / TFOQ26	Aug 2026	Fri **31 Jul 2026**	Fri **24 Jul 2026**	≈ 87	≈ 0.2384

How each row is built:

1. **Futures LTD** — last business day of the month *before* delivery. May 2026 ends on Sun 31 → roll back to Fri 29 May. June 2026 ends on Tue 30 (business day, no roll). July 2026 ends on Fri 31.

2. **Option expiry** — subtract 5 business days from the futures LTD, skipping weekends and TARGET2 holidays.
3. **Days to expiry** — calendar days from reference to option expiry.
4. **T (years)** — `days / 365` (ACT/365).

The exact dates for any reference are available directly from the dashboard — change the **Reference** date picker and the table re-renders for the next 24 contracts.

> **ICE month codes** (used in tickers): F=Jan, G=Feb, H=Mar, J=Apr, **K=May**, **M=Jun**, **N=Jul**, **Q=Aug**, U=Sep, V=Oct, X=Nov, Z=Dec.

SECTION 7 — Troubleshooting

7.1 Common errors and how to fix them

`ModuleNotFoundError: No module named 'black76_ttf'` (or `ttf_market_data`, `structures_ttf`, `ttf_hh_spread`, `ttf_time`)

Cause — Python cannot find the project module on its path. Either the file is missing, or the notebook is being run from a different working directory.

Fix

1. In a terminal, verify the file is next to the notebook: `bash ls dashboard_jupyter.ipynb black76_ttf.py ttf_market_data.py \ structures_ttf.py ttf_hh_spread.py ttf_time.py`
2. If you opened the notebook from a different folder, restart the kernel *from the project folder*: `bash cd /path/to/Options_pricing jupyter notebook dashboard_jupyter.ipynb`
3. As a quick patch, prepend the project folder to `sys.path` in the first cell: `python import sys, os sys.path.insert(0, os.path.abspath(".")) `

`ModuleNotFoundError: No module named 'ipywidgets'` (or numpy, scipy, pandas, matplotlib, plotly)

Cause — a prerequisite is not installed in the kernel you are using.

Fix — install into the **same** Python that runs the notebook:

```
pip install numpy scipy pandas matplotlib ipywidgets plotly
```

If you have several Python installs, run the install from inside the notebook to be sure you hit the right one:

```
import sys
!{sys.executable} -m pip install numpy scipy pandas matplotlib ipywidgets plotly
```

Then *Kernel → Restart*.

Widgets don't display (sliders show as plain text or `Loading widget...`)

Cause — the Jupyter widgets front-end is missing or out of sync with the `ipywidgets` Python package.

Fix

- **Classic Jupyter Notebook**: `bash pip install --upgrade ipywidgets jupyter nbextension enable --py widgetsnbextension`
- **JupyterLab 3+**: `pip install --upgrade ipywidgets jupyterlab` then restart Lab. The widgets manager ships built-in.

- **JupyterLab 2**: `bash jupyter labextension install @jupyter-widgets/jupyterlab-manager`
- **VS Code**: install/update the **Jupyter** extension; in the notebook select … → *Trust Notebook*; pick a kernel that has `ipywidgets` installed (the bottom-right kernel picker shows the path).

After any of the above, restart the kernel and re-run all cells.

Sliders display but charts/output do not refresh

Cause — the `_recompute` / `_refresh` callback raised an exception that was swallowed by `Output()`.

Fix — open the cell's output area and look for a Python traceback. The most common offenders:

- `T = 0` (division by zero): set maturity to a small positive value.
- Vol set to 0: nudge the vol slider to e.g. `0.05`.
- For the spread tab, `ρ = ±1` plus equal vols can produce `σ_spread = 0` and the d1/d2 formula divides by zero — move ρ off the boundary.

`ValueError` / `Brent solver did not converge` in the implied-correlation solver

Cause — the entered market price is outside the feasible Margrabe band for your vol/T inputs (e.g. price below intrinsic, or above the ρ = −1 ceiling).

Fix — check you typed the price in **USD/MMBtu**, not EUR/MWh; verify the vol inputs match the single-name market; widen T if very short-dated.

3D vol surface plot is blank or distorted

Cause — usually a stale matplotlib backend or a strike range that yields invalid log-moneyness near `T = 0`.

Fix

- Restart the kernel.
- Keep `T_max ≥ 0.25` so that `m = ln(K/F)/√T` stays bounded.
- In VS Code, set the matplotlib backend explicitly: `python %matplotlib inline`

Plotly charts (Streamlit dashboard) don't render in the browser

Cause — outdated `plotly` or browser blocking JavaScript.

Fix — `pip install --upgrade plotly`, hard-refresh the page (Ctrl/Cmd + Shift + R), and disable ad-blockers for `localhost`.

Notebook says "Kernel died"

Cause — usually an out-of-memory event caused by repeatedly rebuilding the vol surface on a very fine grid, or a circular widget callback.

Fix — *Kernel → Restart & Clear Output*, then *Run All*. Reduce the strike range / `T_max` before regenerating the surface.

`pip install` works but the new package is not visible in the notebook

Cause — you installed into a different Python than the kernel.

Fix — install from inside the notebook:

```
import sys
!{sys.executable} -m pip install <package>
```

Then *Kernel → Restart*.